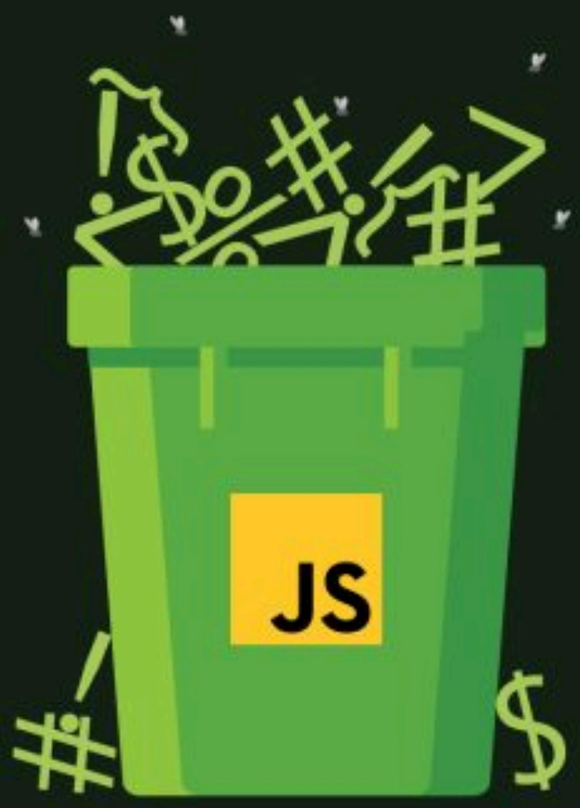

Garbage Collection & Memory Leaks in java script

What is Garbage Collection?

JavaScript automatically manages memory using a process called Garbage Collection.

It removes unused variables and objects from memory when they are no longer reachable in the program.

If nothing is using a piece of memory, JavaScript cleans it up.

This prevents most memory issues — but not all. That’s where Memory Leaks happen.

What is a Memory Leak?

A memory leak happens when:

Memory that is no longer needed is still holding a reference,
so Garbage Collector can’t remove it.

This leads to:

- Slow performance
- Increased RAM usage
- App crashes / browser freeze

Why Memory Leaks Happen

1. Unused Global Variables

```
userData = { name: "John" }; // No let/var/const → becomes global
```

Global variables stay in memory for entire app lifetime.
Fix: Always use let, const, or var.

2. Forgotten Timers & Intervals

```
setInterval(() => {
  console.log("Running...");
}, 1000); global
```

If not cleared, it keeps running forever → memory leak
Fix: clearInterval(id);

Closures Causing Memory Leaks

Closures can hold references to outer variables, keeping them in memory even when not needed.

Problem Example

```
function outer() {
  const largeData = new Array(1000000).fill("data");
  return function inner() {
  console.log("Using closure...");}}
const leak = outer();
```

Here:

- largeData is kept in memory
- Because inner() keeps reference to outer()
- Even if we don’t need it anymore

Fix: Set unused variables to null when done
largeData = null;

Event Listeners Causing Memory Leaks

If you attach an event listener but never remove it, it stays in memory.

Problem Example

```
const button = document.getElementById("btn");
button.addEventListener("click", function() {
  console.log("Clicked");
});
```

Even if the button is removed from the DOM, the memory may remain.

Fix: button.removeEventListener("click", handler);

Always clean up listeners when removing elements.

Real-world Example (React)

Bad Practice:

```
useEffect(() => {
  window.addEventListener("resize", handleResize);
});
```

Good Practice:

```
useEffect(() => {
  window.addEventListener("resize", handleResize);
 return () => {
    window.removeEventListener("resize", handleResize);
  };
}, []);
```

This prevents memory leaks in React apps

How To Prevent Memory Leaks

- Use let and const
- Clear intervals / timeouts
- Remove unused event listeners
- Set big unused variables to null
- Be careful with closures
- Use cleanup functions in React

LinkedIn divyanshi-pal